

ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA DE SOFTWARE



Tema:

Diseño arquitectónico de la aplicación: asistente conversacional HelizLlama para Moodle basado en RAG con modelos locales

Integrantes:

Jairo Jhoel Ramírez Cevallos

Brandon Mateo Acurio Veloz

Asignatura:

Aplicaciones Informáticas II

Docente:

Ing. Julio Santillan

Curso:

Octavo

Período académico:

Septiembre 2026 – febrero 2027

Riobamba, 10 de octubre de 2026

1. Introducción

El asistente HelizLlama adopta una arquitectura cliente-servidor distribuida en capas, formada por dos subsistemas: un plugin de Moodle (PHP) que actúa como capa de presentación e integración, y un servicio de orquestación RAG (Python) organizado internamente con arquitectura hexagonal. El orquestador se comunica por REST/JSON con el servicio de inferencia local (Ollama) y por SQL con la base de datos vectorial (PostgreSQL con la extensión pgvector).

- **Propósito.** Este documento define la estructura del sistema, los estilos y patrones adoptados, la comunicación entre capas, la vista de despliegue y la justificación de cada decisión frente a los requisitos.
- **Alcance.** El diseño cubre los cuatro módulos funcionales definidos en la ERS: consulta conversacional, gestión del conocimiento, administración y seguimiento docente. La evaluación con RAGAS queda fuera del producto; sin embargo, la arquitectura conserva la pregunta, el contexto recuperado y la respuesta para que pueda ejecutarse (RNF-10).
- **Audiencia.** Equipo de desarrollo (codificación, integración y pruebas), docente de la asignatura (verificación de los objetivos arquitectónicos) y administradores de Moodle de la ESPOCH (despliegue y operación).
- **Documento de referencia.** Los códigos RF, RNF y CU, así como las secciones citadas (2.4, 3.1.1, etc.), corresponden a la Especificación de Requisitos de Software (ERS) del proyecto, versión [completar].

2. Descripción del sistema

HelizLlama es un asistente conversacional integrado en los cursos de Moodle que responde las preguntas de los estudiantes usando únicamente el contenido de cada curso. Aplica generación aumentada por recuperación (RAG) con modelos de pesos abiertos ejecutados localmente (Mistral 7B para la generación y BGE-M3 para los embeddings), sin enviar datos a servicios en la nube.

El sistema atiende a tres actores, cuyos roles y permisos se heredan de Moodle:

- **Estudiante:** consulta al asistente desde el panel del curso y valora cada respuesta como útil o no útil.
- **Docente:** activa el asistente en su curso, revisa el estado del índice y consulta informes de uso con registros disociados.
- **Administrador:** configura el plugin a nivel de sitio y los parámetros de indexación, y supervisa la salud del orquestador.

2.1. Módulos funcionales

Tabla 1. Módulos funcionales del sistema

Módulo	Requisitos	Función
Consulta conversacional	RF-01 a RF-09	Recibe la pregunta, recupera fragmentos del curso, genera la respuesta con citas de las fuentes y registra la interacción de forma disociada.
Gestión del conocimiento	RF-10 a RF-13	Extrae, fragmenta, vectoriza e indexa los recursos del curso y mantiene el índice actualizado ante cambios.
Administración	RF-14 a RF-16	Configura modelos, umbrales y parámetros, y verifica el estado de los servicios.
Seguimiento docente	RF-17, RF-18	Presenta al docente informes de uso del asistente en su curso.

Fuente: ERS del proyecto. Realizado por: Ramírez J.; Acurio B., 2026.

3. Requisitos arquitectónicos significativos

Doce requisitos de la ERS condicionan la forma del sistema; cada uno produce una decisión arquitectónica concreta, como se resume en la Tabla 2.

Tabla 2. Requisitos arquitectónicos significativos

Driver	Requisitos de origen	Consecuencia arquitectónica
Integración sin modificar el núcleo de Moodle	Restricción 2.4; RNF-14	El front-end del asistente es un plugin (tipos local y block) que usa solo las API de Moodle.
Lenguajes impuestos: PHP y Python	Restricción 2.4	Dos subsistemas separados que se comunican por red.
Modelos de pesos abiertos y ejecución local	Restricción 2.4; RNF-18	Servicio de inferencia local (Ollama) detrás de una interfaz intercambiable.
Hardware limitado (GPU de 10 GB de VRAM)	Restricción 2.4; RNF-01; RNF-03	Un único punto de inferencia con cola de peticiones y concurrencia acotada.
Respuestas ancladas al curso y aislamiento	RF-03; RF-04; RF-05; RNF-06	Filtro obligatorio por curso en la recuperación y umbral de relevancia antes de generar.

Driver	Requisitos de origen	Consecuencia arquitectónica
Privacidad (LOPDP) y solo lectura	RF-07; RNF-05; RNF-07; RNF-09	Registro disociado, filtrado de contenido sensible en la extracción y capacidades de solo lectura.
Seguridad de la comunicación	RNF-08	HTTPS y token entre plugin y orquestador; inferencia y base de datos en red privada.
Tolerancia a fallos	RF-09; RNF-12	Llamadas con tiempo máximo y degradación controlada; Moodle nunca depende del asistente.
Integridad del índice	RF-10; RF-11; RNF-11	Indexación transaccional con versiones del índice por curso.
Actualización del conocimiento	RF-11; RNF-15	Eventos y tareas programadas de Moodle disparan la reindexación incremental.
Portabilidad	RNF-17	Orquestador, inferencia y base vectorial en contenedores Docker.
Exactitud medible con RAGAS	RNF-10	El orquestador conserva consulta, contexto y respuesta para la evaluación.

Fuente: ERS del proyecto. Realizado por: Ramírez J.; Acurio B., 2026.

4. Arquitectura seleccionada

El estilo principal es cliente-servidor distribuido en capas; dentro de él se aplican cinco estilos complementarios, cada uno en un nivel distinto del sistema (Tabla 3).

Tabla 3. Estilos arquitectónicos por nivel

Nivel	Estilo adoptado	Dónde se aplica
Sistema completo	Cliente-servidor distribuido en capas (presentación, aplicación, servicios de IA, persistencia)	Navegador → Moodle/plugin → orquestador RAG → Ollama y PostgreSQL
Integración con Moodle	Microkernel (núcleo + plugins)	El plugin extiende Moodle a través de sus API, como cualquier plugin de tipo local o block.

Nivel	Estilo adoptado	Dónde se aplica
Comunicación entre subsistemas	Orientado a servicios con API REST/JSON	Plugin ↔ orquestador; orquestador ↔ Ollama
Interior del orquestador	Hexagonal (puertos y adaptadores)	El dominio RAG no conoce Ollama, pgvector ni Moodle; solo interfaces.
Indexación	Tuberías y filtros (pipeline)	Extraer → limpiar → fragmentar → vectorizar → almacenar
Actualización del índice	Dirigido por eventos	Observadores de eventos de Moodle y tareas programadas

Realizado por: Ramírez J.; Acurio B., 2026.

El sistema se compone de cinco elementos desplegables, distribuidos en tres nodos (cliente, servidor Moodle y estación de IA):

1. **Plugin HelizLlama** (local_helizllama y block_helizllama), instalado en el servidor Moodle existente; incluye el panel de chat del curso.
2. **Orquestador RAG** en Python (FastAPI), que recibe las peticiones del plugin y coordina la recuperación y la generación.
3. **Servicio de inferencia Ollama**, que ejecuta Mistral 7B y BGE-M3 en la GPU.
4. **PostgreSQL con pgvector**, base de datos vectorial con fragmentos, vectores y versiones del índice.
5. **Base de datos de Moodle**, donde el plugin guarda su configuración, el estado del índice y los registros disociados.

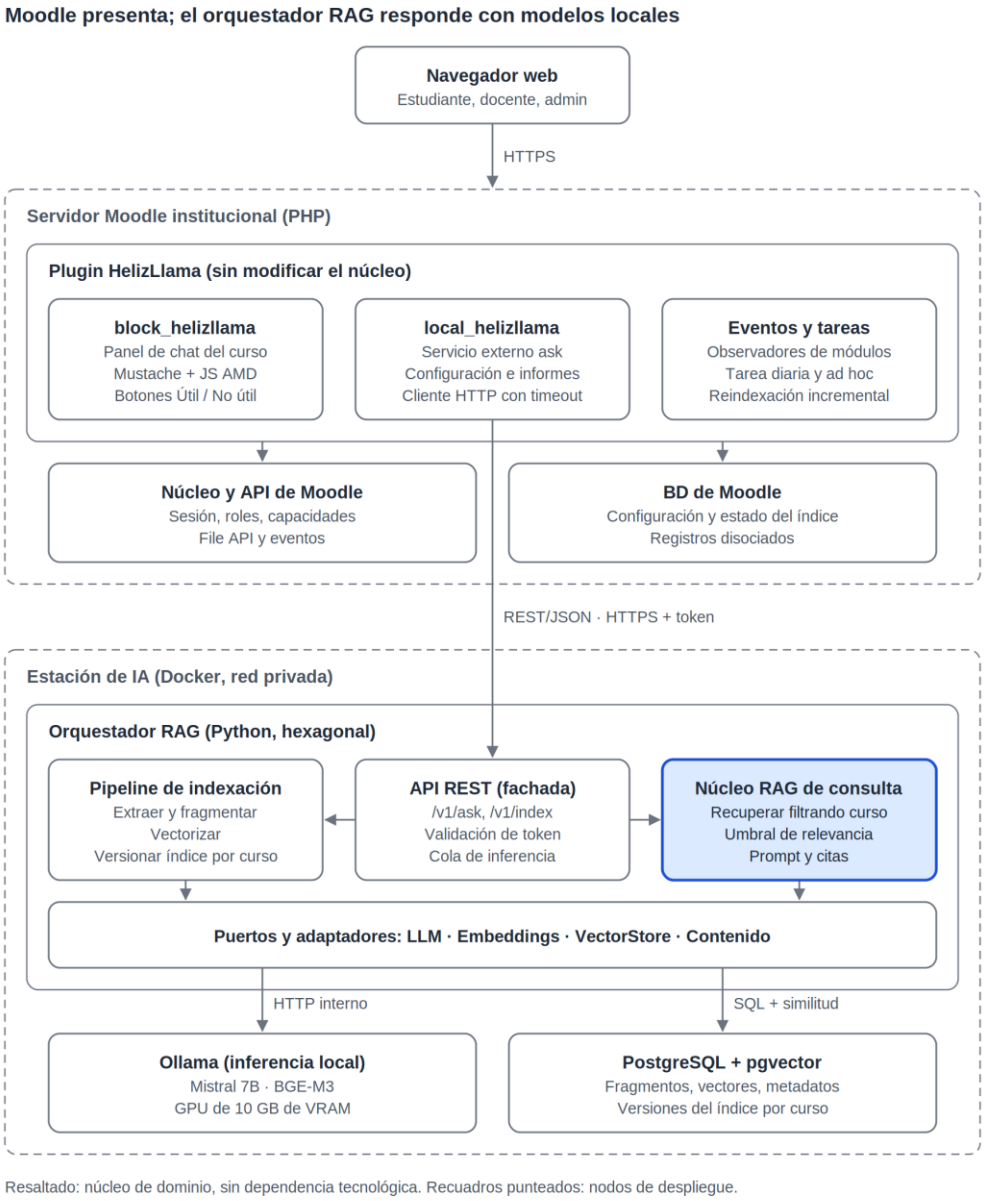
Los elementos 2, 3 y 4 se ejecutan como contenedores Docker en la estación de IA; los elementos 1 y 5 forman parte de la instalación institucional de Moodle.

5. Diagramas arquitectónicos

5.1. Vista general del sistema

El usuario solo interactúa con Moodle: el plugin valida la sesión y los permisos, llama al orquestador, y esta consulta a Ollama y a la base vectorial. El núcleo RAG de consulta, resaltado en la Ilustración 1, es el dominio que no depende de ninguna tecnología concreta.

Ilustración 1. Diagrama arquitectónico general de HelizLlama



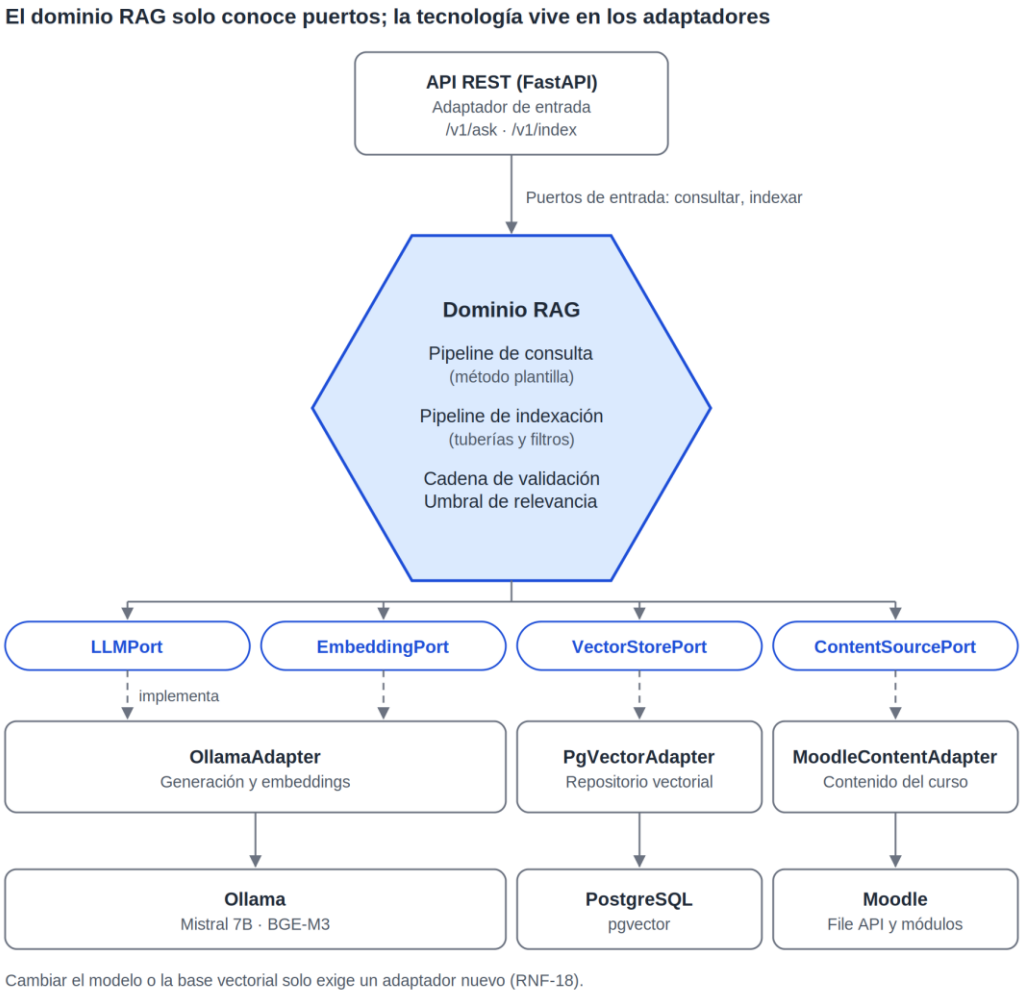
Resaltado: núcleo de dominio, sin dependencia tecnológica. Recuadros punteados: nodos de despliegue.

Realizado por: Ramírez J.; Acurio B., 2026.

5.2. Vista interna del orquestador (arquitectura hexagonal)

El dominio RAG se programa contra cuatro puertos de salida (LLMPort, EmbeddingPort, VectorStorePort y ContentSourcePort) y expone dos puertos de entrada (consultar e indexar) a través de la API REST. Las tecnologías concretas quedan en los adaptadores, por lo que pueden sustituirse sin modificar el dominio.

Ilustración 2. Arquitectura hexagonal del orquestador RAG



Realizado por: Ramírez J.; Acurio B., 2026.

5.3. Responsabilidad de cada componente

Tabla 4. Componentes, responsabilidades y trazabilidad

Componente	Responsabilidad	RF / RNF
block_helizllama	Panel conversacional, historial de la sesión, indicador de espera, botones Útil / No útil	RF-02, RF-08; 3.1.1
local_helizllama	Servicio externo ask, control de capacidades, configuración del sitio y por curso, informes docentes, cliente HTTP con timeout	RF-01, RF-06, RF-07, RF-09, RF-14 a RF-18
Eventos y tareas	Observadores de módulos de curso, tarea programada diaria y tarea ad hoc de indexación	RF-11, RF-12, RNF-15

Componente	Responsabilidad	RF / RNF
Núcleo y BD de Moodle	Sesión, roles, File API; tablas propias del plugin para configuración, estado del índice y registros disociados	RF-13, RNF-05, RNF-09
API REST (fachada)	Endpoints /v1/ask, /v1/index, /v1/health y /v1/models; autenticación por token; cola de inferencia	RNF-01, RNF-03, RNF-08
Núcleo RAG de consulta	Vectorizar la pregunta, recuperar top-k filtrado por curso, aplicar el umbral, construir el prompt y adjuntar las fuentes	RF-03, RF-04, RF-05, RNF-06
Pipeline de indexación	Extraer texto (página, PDF, DOCX), fragmentar, vectorizar y escribir una nueva versión del índice	RF-10, RF-11, RNF-04, RNF-11
Puertos y adaptadores	Interfaces del dominio y sus implementaciones para Ollama, pgvector y el contenido de Moodle	RNF-18
Ollama	Ejecuta Mistral 7B (generación) y BGE-M3 (embeddings) en la GPU	Restricción 2.4
PostgreSQL + pgvector	Fragmentos, vectores, metadatos de curso y recurso, versiones del índice	RF-03, RNF-02

Realizado por: Ramírez J.; Acurio B., 2026.

6. Comunicación entre capas

El sistema usa cuatro tramos de comunicación, y solo uno de ellos sale de la estación de IA: el que une el plugin con el orquestador (Tabla 5).

Tabla 5. Mecanismos de comunicación por tramo

Tramo	Protocolo y formato	Seguridad	Control de fallos
Navegador → Moodle (plugin)	HTTPS; plantillas Mustache y llamadas AJAX mediante módulos JS AMD a servicios externos de Moodle	Sesión y capacidades de Moodle	Si el asistente no responde, el curso sigue operativo.

Tramo	Protocolo y formato	Seguridad	Control de fallos
Plugin → orquestador	REST/JSON sobre HTTPS (/v1/ask, /v1/index, /v1/health, /v1/models)	Token en la cabecera de autorización; puerto publicado solo hacia el servidor Moodle	Cliente HTTP con tiempo máximo; mensaje claro al usuario y registro del incidente.
Orquestador → Ollama	HTTP interno con JSON (API de Ollama)	Red interna de Docker, sin puerto expuesto	Cola de inferencia con concurrencia acotada y tiempo máximo.
Orquestador → PostgreSQL + pgvector	SQL con búsqueda por similitud vectorial	Red interna de Docker; credenciales por variables de entorno	Indexación transaccional; si falla, sigue vigente la versión anterior del índice.

Realizado por: Ramírez J.; Acurio B., 2026.

El tramo plugin → orquestador es sin estado: cada petición incluye el token y el identificador del curso, y la fachada valida ambos antes de llegar al dominio. Toda consulta a la base vectorial pasa por el repositorio, que exige course_id, de modo que el aislamiento entre cursos no depende del código que hace la llamada.

7. Vista de despliegue y flujos principales

7.1. Vista de despliegue

El sistema se despliega en tres nodos y el servidor Moodle institucional no requiere hardware adicional. Solo el contenedor del orquestador publica un puerto hacia Moodle (Ilustración 3).

Ilustración 3. Diagrama de despliegue



Realizado por: Ramírez J.; Acurio B., 2026.

Tabla 6. Nodos de despliegue

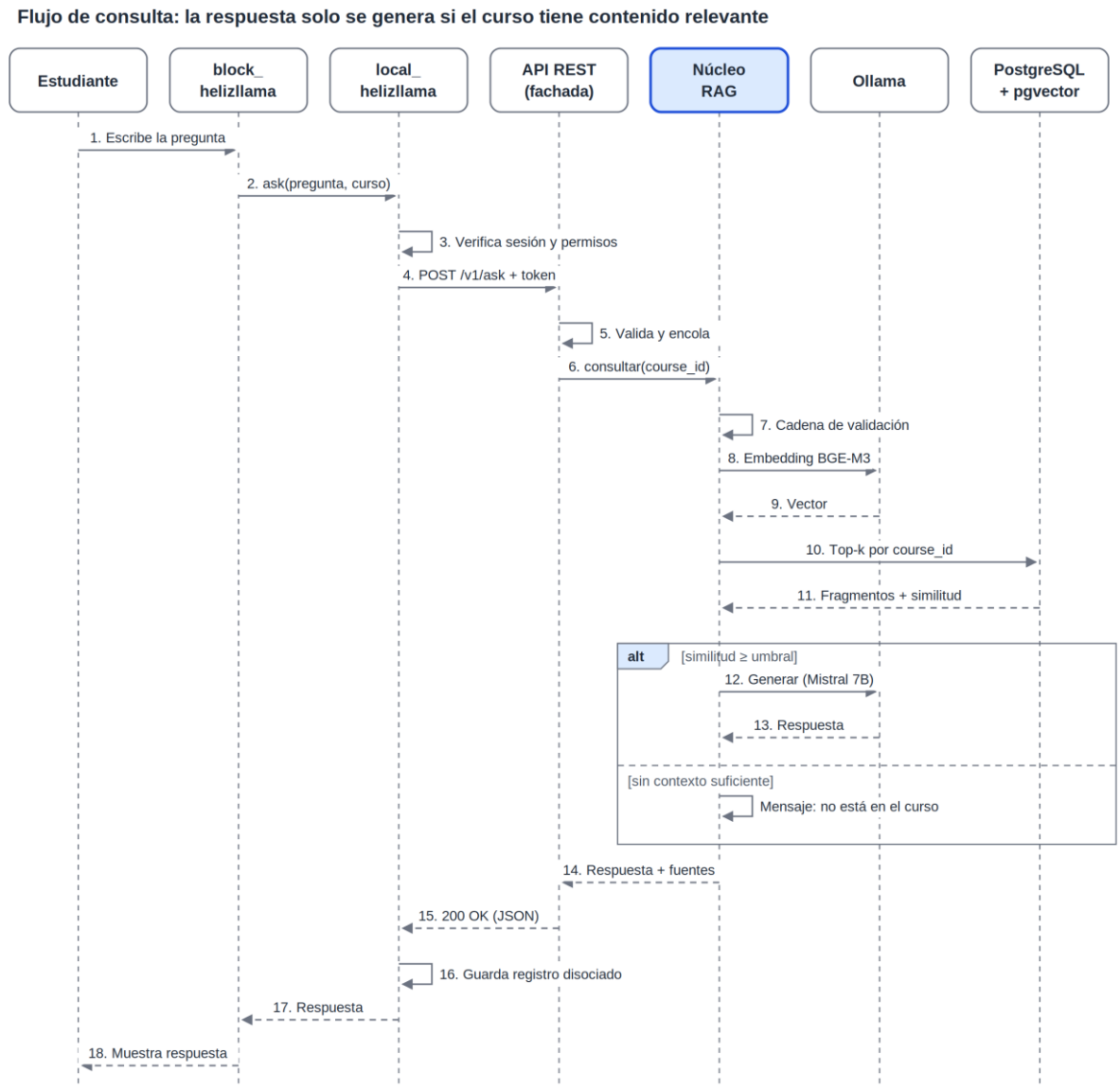
Nodo	Componentes	Red	Requisitos
Cliente	Navegador web	Internet o red ESPOCH, HTTPS	Interfaces 3.1.2
Servidor Moodle (institucional)	Moodle + local_helizllama + block_helizllama; BD de Moodle	Expuesto por HTTPS	Sin hardware adicional (3.1.2)
Estación de IA (Linux, Docker Compose, GPU RTX de 10 GB, 32 GB de RAM)	Contenedores rag-orchestrator (FastAPI), ollama (Mistral 7B, BGE-M3) y postgres-pgvector	Solo el orquestador publica un puerto HTTPS hacia el servidor Moodle; Ollama y PostgreSQL quedan en la red interna de Docker	RNF-08; RNF-17

Realizado por: Ramírez J.; Acurio B., 2026.

7.2. Flujo de consulta

La Ilustración 4 muestra el flujo principal del sistema: una pregunta del estudiante solo llega al modelo generativo si el curso contiene fragmentos que superan el umbral de relevancia.

Ilustración 4. Diagrama de secuencia del flujo de consulta



Realizado por: Ramírez J.; Acurio B., 2026.

1. El estudiante escribe una pregunta en el panel block_helizllama del curso.
2. local_helizllama verifica la sesión y la capacidad del usuario en el curso, y envía la pregunta con el course_id al orquestador (POST /v1/ask).
3. La fachada valida el token y el tamaño de la consulta y la coloca en la cola de inferencia.

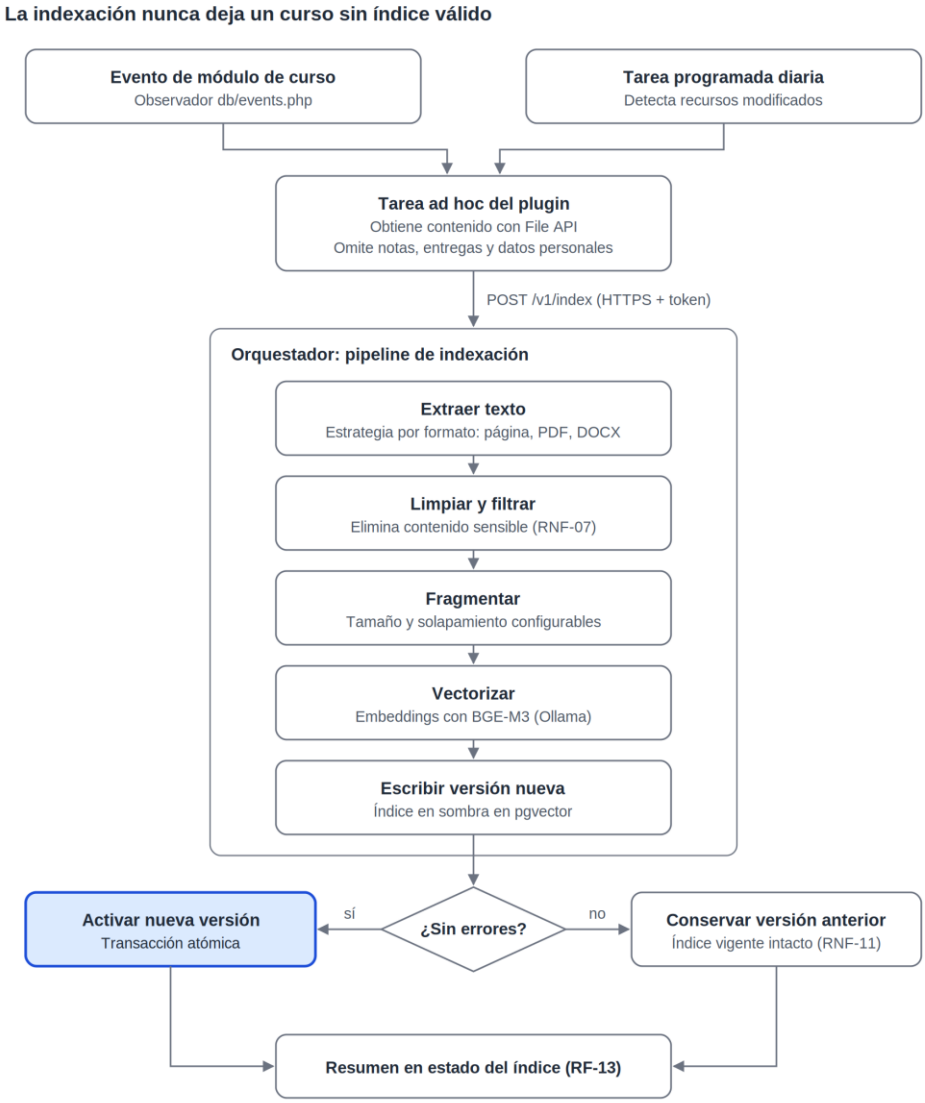
4. La cadena de validación descarta preguntas fuera de alcance (notas, datos personales) y responde sin invocar al modelo.
5. El núcleo RAG vectoriza la pregunta con BGE-M3 y recupera los top-k fragmentos filtrados por course_id.
6. Si ningún fragmento supera el umbral de relevancia, el sistema indica que la información no está en el curso; si lo supera, construye el prompt y Mistral 7B genera la respuesta.
7. El orquestador devuelve la respuesta con las fuentes citadas y conserva consulta, contexto y respuesta para RAGAS (RNF-10).
8. El plugin muestra la respuesta, guarda el registro disociado y recoge la valoración Útil / No útil.

Si en cualquier paso se excede el tiempo máximo, el plugin muestra un mensaje claro y registra el incidente (RF-09, RNF-12).

7.3. Flujo de indexación (CU-07, CU-08)

La indexación se ejecuta en segundo plano y escribe siempre una versión nueva del índice, de modo que un fallo nunca deja al curso sin un índice válido (Ilustración 5).

Ilustración 5. Flujo de indexación con versionado del índice



Realizado por: Ramírez J.; Acurio B., 2026.

1. Un evento de módulo de curso o la tarea programada diaria marca los recursos nuevos, modificados o eliminados.
2. Una tarea ad hoc del plugin obtiene el contenido visible con la File API y las API de módulos, omitiendo calificaciones, entregas y datos personales (RNF-07).
3. El plugin envía los recursos al orquestador (POST /v1/index).
4. El pipeline extrae el texto, fragmenta según el tamaño y el solapamiento configurados, vectoriza con BGE-M3 y escribe en una nueva versión del índice del curso.
5. Si termina sin errores, la nueva versión se activa en una transacción; si falla, se conserva la anterior (RNF-11) y el resumen aparece en el estado del índice (RF-13).

8. Justificación de estilos y patrones

Cada estilo y patrón responde a un requisito o restricción de la ERS; ninguno se adoptó solo por preferencia tecnológica.

8.1. Estilos arquitectónicos

- **Cliente-servidor distribuido en capas.** La ERS impone PHP para el plugin y Python para el orquestador, y el modelo debe ejecutarse en una estación con GPU distinta del servidor Moodle. Separar ambos lados permite desplegarlos en máquinas diferentes y aislar fallos: si cae la inferencia, Moodle sigue operando (RNF-12). Cada capa llama solo a la inmediata inferior, lo que simplifica pruebas y mantenimiento (RNF-14).
- **Microkernel (plugins de Moodle).** Es el estilo nativo de Moodle: un núcleo estable con API comunes y plugins que lo extienden. Adoptarlo cumple la restricción de no modificar el núcleo y permite heredar sesión, roles, capacidades y contexto de curso sin re implementarlos (RF-01, RNF-05). El plugin local contiene la lógica, los servicios externos, las tareas y los eventos; el plugin block, el panel del curso.
- **Orientado a servicios con REST/JSON.** Es el mecanismo previsto en la ERS (3.1.3 y 3.1.4) y el que expone Ollama. Un contrato REST desacopla PHP de Python, permite autenticar con token sobre HTTPS (RNF-08) y deja abierta la integración con la Moodle App, que también consume servicios web.
- **Hexagonal dentro del orquestador.** El núcleo RAG se programa contra puertos (LLMPort, EmbeddingPort, VectorStorePort, ContentSourcePort), y Ollama y pgvector son adaptadores. Cambiar Mistral por otro modelo o pgvector por otra base no toca el dominio (RNF-18, suposición 2.5), y el dominio puede probarse con adaptadores falsos.
- **Tuberías y filtros en la indexación.** La indexación es una secuencia lineal de transformaciones con entradas y salidas definidas. Modelarla como pipeline permite añadir filtros, como OCR (evolución 2.6), sin reescribir los demás, y aislar en un filtro dedicado la exclusión de datos personales y calificaciones (RNF-07).
- **Dirigido por eventos.** Moodle emite eventos al crear, modificar o eliminar módulos de curso. Observarlos permite marcar recursos pendientes de reindexar sin bloquear la acción del docente, y la tarea programada diaria procesa los cambios (RF-11, RNF-15).

8.2. Patrones de diseño

Tabla 7. Patrones de diseño y su justificación

Patrón	Componente	Requisito	Justificación
Fachada (API Gateway ligero)	API REST del orquestador	RNF-08	Punto único de entrada que valida token, tamaño de la consulta y curso antes de llegar al dominio.
Adaptador	OllamaAdapter, PgVectorAdapter, MoodleContentAdapter	RNF-18; suposición 2.5	Traduce las API externas a los puertos del dominio.
Estrategia	Fragmentación y extracción por formato (página, PDF, DOCX)	RF-10; RF-15	Tamaño, solapamiento y extractor se eligen en tiempo de ejecución según configuración y tipo de recurso.
Repositorio	Acceso a fragmentos y registros	RF-03; RF-07; RF-13	Oculta el SQL y la búsqueda por similitud; obliga a pasar course_id en toda consulta (RNF-06).
Cadena de responsabilidad	Validación y enrutamiento de la consulta	RF-02; RF-05; RF-06	Valida longitud, detecta preguntas fuera de alcance y aplica el umbral antes de invocar al LLM.
Método plantilla (Template Method)	Pipeline RAG de consulta	RF-03; RF-04	Pasos fijos (vectorizar, recuperar, filtrar, construir prompt, generar, citar) con implementaciones intercambiables.
Observador	db/events.php del plugin	RF-11	Reacciona a eventos de módulo de curso sin acoplarse al código que los emite.
Productor-consumidor (cola)	Cola de inferencia del orquestador	RNF-01; RNF-03	Las peticiones exceden la capacidad de una GPU de 10 GB; se encolan con concurrencia acotada en lugar de rechazarse.

Patrón	Componente	Requisito	Justificación
Timeout y degradación controlada	Cliente HTTP plugin → orquestador → Ollama	RF-09; RNF-12	Al exceder el tiempo máximo se devuelve un mensaje claro y se registra el incidente.
Versionado del índice (escritura en sombra)	Indexación por curso	RNF-11; RF-10	Se construye una nueva versión y se activa al final en una transacción; si falla, sigue la anterior.
MVC con renderers y plantillas Mustache	Panel y páginas de administración del plugin	Interfaces 3.1.1	Sigue las guías de Moodle, respeta el tema institucional y separa la vista de la lógica.
Inyección de dependencias	Arranque del orquestador	RNF-14; RNF-18	Los adaptadores se eligen por configuración, sin cambiar código.

Realizado por: Ramírez J.; Acurio B., 2026.

9. Tecnologías

Tabla 8. Tecnologías por capa

Tecnología	Capa	Uso en el sistema
Moodle (PHP)	Presentación e integración	Plataforma anfitriona; aporta sesión, roles, File API, eventos y tareas programadas.
Mustache + JavaScript AMD	Presentación	Plantillas y comportamiento del panel de chat y de las páginas del plugin.
Python + FastAPI	Aplicación	Orquestador RAG: API REST, cola de inferencia, núcleo de dominio y pipeline de indexación.
Ollama	Servicios de IA	Servidor de inferencia local para los modelos.
Mistral 7B (cuantizado Q4)	Servicios de IA	Generación de respuestas.
BGE-M3	Servicios de IA	Generación de embeddings de preguntas y fragmentos.

Tecnología	Capa	Uso en el sistema
PostgreSQL + pgvector	Persistencia	Almacenamiento de fragmentos, vectores, metadatos y versiones del índice.
Docker Compose	Despliegue	Empaqueta orquestador, Ollama y base vectorial en la estación de IA (RNF-17).
HTTPS + token	Seguridad	Cifrado y autenticación del tramo plugin → orquestador (RNF-08).
RAGAS	Evaluación (fuera del producto)	Medición de exactitud con los datos que conserva el orquestador (RNF-10).

Realizado por: Ramírez J.; Acurio B., 2026.

10. Alternativas, trazabilidad y riesgos

10.1. Alternativas descartadas

Tabla 9. Alternativas evaluadas

Alternativa	Motivo del descarte
Monolito: toda la lógica RAG dentro del plugin PHP	Viola la restricción de usar Python para el orquestador; el ecosistema RAG (embeddings, extracción de PDF y DOCX) es más maduro en Python; acopla la GPU al servidor Moodle.
Microservicios finos (indexación, consulta y métricas por separado)	Complejidad operativa desproporcionada para 2 desarrolladores, 16 semanas y 20 usuarios concurrentes (RNF-03); la separación se conserva como módulos internos del orquestador, divisibles en el futuro.
Serverless o APIs de IA en la nube	Prohibido por la restricción de modelos locales sin costo de licencia y por la LOPDP.
Consumo directo de Ollama desde el plugin	Dejaría recuperación, umbral y construcción del prompt en PHP, expondría Ollama fuera de la red privada (RNF-08) y ataría el plugin a un proveedor de inferencia (RNF-18).
Indexación síncrona en la petición del docente	Una indexación de hasta 15 minutos (RNF-04) bloquearía la interfaz; se usa una tarea ad hoc en segundo plano.

Realizado por: Ramírez J.; Acurio B., 2026.

10.2. Trazabilidad módulo-componente

Tabla 10. Trazabilidad entre módulos y componentes

Módulo de la ERS	RF	Componentes principales
Consulta conversacional	RF-01 a RF-09	Bloque de chat, servicio externo ask, API REST, cadena de validación, pipeline RAG, cola de inferencia, registro disociado
Gestión del conocimiento	RF-10 a RF-13	Observadores de eventos, tareas programadas, adaptador de contenido Moodle, pipeline de indexación, repositorio vectorial
Administración	RF-14 a RF-16	Páginas de configuración del plugin, endpoints de salud y modelos del orquestador
Seguimiento docente	RF-17, RF-18	Informes del plugin sobre la tabla de registros disociados

Realizado por: Ramírez J.; Acurio B., 2026.

10.3. Riesgos y mitigaciones

Tabla 11. Riesgos y mitigaciones

Riesgo	Mitigación arquitectónica
Mistral 7B y BGE-M3 compiten por 10 GB de VRAM	Modelo cuantizado (Q4) y cola con concurrencia acotada; medir RNF-01 desde el primer prototipo.
La versión institucional de Moodle aún no está confirmada	El plugin usa solo API estables (servicios externos, eventos, tareas, File API) y se valida en un Moodle de pruebas en Docker.
Un reintento de indexación deja duplicados	Indexación idempotente por course_id + cmid + hash del contenido.
Fuga de contenido entre cursos	Filtro por course_id impuesto en el repositorio, no en el llamador, y pruebas automáticas de aislamiento.
Caída de la estación de IA	Degradación controlada: el panel informa que el asistente no está disponible y Moodle sigue operando (RNF-12).

Realizado por: Ramírez J.; Acurio B., 2026.

11. Conclusiones

- La arquitectura cliente-servidor distribuida en capas cumple las restricciones de la ERS: separa el plugin PHP del orquestador Python, mantiene la inferencia en la red privada y garantiza que Moodle siga operando, aunque el asistente falle.
- El estilo microkernel permite integrar el asistente sin modificar el núcleo de Moodle y heredar sesión, roles y contexto de curso, lo que reduce el código propio y el riesgo de seguridad.
- La arquitectura hexagonal del orquestador aísla el dominio RAG de Ollama, pgvector y Moodle, de modo que el modelo o la base vectorial pueden cambiarse sin reescribir la lógica de consulta.
- El filtro obligatorio por course_id en el repositorio, el registro disociado y la exclusión de datos personales en el pipeline responden a los requisitos de aislamiento y de la LOPDP desde la propia estructura, no solo desde la interfaz.
- La cola de inferencia, el modelo cuantizado y el versionado del índice hacen viable el sistema con una sola GPU de 10 GB y protegen la integridad del conocimiento indexado.

Referencias

Bass, L., Clements, P. y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Cockburn, A. (2005). *Hexagonal architecture (ports and adapters)*. alistair.cockburn.us.

Es, S., James, J., Espinosa-Anke, L. y Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. *Proceedings of EACL 2024: System Demonstrations*.

Lewis, P., Perez, E., Piktus, A. et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*.

Moodle HQ. (s. f.). *Moodle developer resources: plugin types, Events API, Task API, external services*. moodledev.io.

Richards, M. y Ford, N. (2020). *Fundamentals of software architecture*. O'Reilly Media.

[Agregar la ERS del proyecto con su versión y fecha.]